Práctica 1b: Despliegue de frontend y backend en dos instancias EC2

1. Objetivo

Separar en dos instancias la aplicación desplegada en la Práctica 1a. Se conservará la instancia EC2 creada en esa práctica como frontend y se creará una segunda instancia para el backend:

- **Frontend existente**: Apache y JavaScript; recibe el tráfico público y sirve la web.
- **Backend**: PHP y MySQL; procesa las peticiones de la aplicación y aloja la base de datos.

La base de datos no tendrá acceso público. Las peticiones seguirán el recorrido:

```
Navegador → Apache (frontend) → PHP (backend) → MySQL (backend)
```

2. Arquitectura y recursos

1. Conservad la EC2 de la Práctica 1a como `frontend`. No la terminéis ni la reemplacéis: mantendrá su dirección IP elástica, DNS, Apache y certificado HTTPS.
2. Cread una nueva EC2 Ubuntu Server como `backend`, en la misma VPC y región. **No le asignéis IP pública.**
3. Anotad la IP privada del backend. Apache en el frontend reenviará a esa dirección las rutas de la API.
4. Trasladad al backend PHP, MySQL, la configuración de la aplicación y la base de datos. En el frontend quedarán Apache y los archivos estáticos HTML, CSS y JavaScript.

Reutilizad el código y los datos de prueba que ya están desplegados.

3. Grupos de seguridad

Aplicad mínimo privilegio. Asociad cada grupo a la instancia correspondiente.

Grupo / instancia	Protocolo y puerto	Origen permitido
Frontend	TCP/80 (HTTP)	Público (`0.0.0.0/0`)
Frontend	TCP/443 (HTTPS)	Público (`0.0.0.0/0`)
Frontend	TCP/22 (SSH)	Vuestra IP pública o el origen autorizado por el laboratorio
Backend	TCP/80 (API/PHP)	**Solo el grupo de seguridad del frontend**
Backend	TCP/22 (SSH)	Solo el grupo del frontend, si administráis el backend mediante salto SSH
Backend	TCP/3306 (MySQL)	**Sin regla de entrada**

Cómo usar un grupo de seguridad como origen

Un grupo de seguridad es un conjunto de reglas de firewall asociado a las interfaces de red de las instancias; no es una ACL ni una dirección IP. Al usar el grupo del frontend como origen, AWS permite el tráfico que llega desde las interfaces de red asociadas a ese grupo, siempre que coincidan el protocolo y el puerto de la regla. No hay que introducir la IP privada del frontend: AWS mantiene esta referencia aunque cambie esa IP.

Para configurar la regla de entrada en el grupo del backend:

1. En la consola de AWS, abrid **EC2** → **Grupos de seguridad** y seleccionad el grupo asociado a la instancia `backend`.
2. Elegid **Reglas de entrada** → **Editar reglas de entrada** → **Añadir regla**.
3. Seleccionad tipo **HTTP** (TCP, puerto 80). En **Origen**, elegid **Personalizado** y buscad el ID (`sg-...`) o la etiqueta **Name** del grupo asociado a la instancia `frontend`; seleccionad el grupo correcto de la lista.
4. Guardad las reglas y comprobad que el origen de la regla TCP/80 aparece como el grupo del frontend, no como una dirección o rango CIDR.

Si ya existe una regla TCP/80 con origen CIDR, añadid primero la nueva regla con origen de grupo de seguridad. Después eliminad la regla CIDR anterior si ya no debe permitirse ese acceso. No intentéis convertir una regla CIDR existente cambiando su origen: AWS no permite combinar ambos tipos de origen en una misma regla.

Las ACL de red (NACL) son distintas: se aplican a nivel de subred y sus reglas usan direcciones IP o rangos CIDR. En esta práctica, la restricción entre frontend y backend se configura en el grupo de seguridad del backend.

4. Despliegue y configuración

4.1 Crear y preparar el backend

1. Instalad PHP, las extensiones requeridas por la aplicación, Apache con PHP y MySQL.

2. Desplegad en el backend los archivos PHP y la configuración de la aplicación. Apache/PHP atenderá la API en el puerto 80 y solo aceptará tráfico del grupo de seguridad frontend.

3. Configurad MySQL para escuchar únicamente en loopback, por ejemplo:

   ```
   bind-address = 127.0.0.1
   ```

   Aplicad el cambio y comprobad que MySQL escucha en `127.0.0.1`, no en todas las interfaces.

4. Cread la base de datos de la aplicación y un usuario específico con permisos solo sobre esa base de datos. Limitad el usuario a conexiones locales, por ejemplo `'app_user'@'localhost'`. No uséis `root` desde PHP ni habilitéis un usuario remoto.

5. Configurad PHP para conectarse a MySQL mediante `localhost` o `127.0.0.1`. Guardad la contraseña en un archivo de configuración no público, con permisos restringidos y excluido del

repositorio.

6. Exportad la base de datos que utiliza la instancia de la Práctica 1a, trasladad el volcado al backend e importadlo en la nueva base de datos. Proteged y eliminad el archivo temporal cuando hayáis comprobado la migración.

4.2 Adaptar el frontend existente

1. Mantened la instancia, Apache, los archivos estáticos, el DNS y el certificado HTTPS existentes.
2. Conservad en el frontend únicamente los archivos HTML, CSS y JavaScript. Retirad la aplicación PHP y, una vez comprobada la migración, PHP y MySQL y sus configuraciones asociadas. No borréis los datos originales hasta verificar que el backend funciona.
3. Configurad Apache como proxy inverso para reenviar las rutas de la API a la IP privada del backend por HTTP.
4. Ajustad la configuración de la aplicación para que las peticiones JavaScript utilicen las rutas del frontend. El navegador no debe conectarse directamente al backend ni a MySQL; no expongáis la IP privada del backend en el código cliente.
5. Comprobad que Apache frontend sigue atendiendo el nombre DNS por HTTPS y que una operación de la aplicación llega a PHP en el backend y a MySQL.

4.3 DNS, IP elástica y HTTPS

No cambiéis la asociación de la IP elástica ni la entrada `ADDRESS` de `yatat.es`: deben continuar apuntando a la EC2 frontend de la Práctica 1a. No creéis ni asociéis otra IP elástica. Comprobad que el DNS y el certificado HTTPS existentes siguen funcionando después de configurar el proxy inverso.

5. Comprobaciones de aceptación

- ☐ La EC2 de la Práctica 1a sigue siendo el frontend y conserva su IP elástica, DNS y certificado.
- ☐ Se ha creado una EC2 backend sin IP pública.
- ☐ Solo el frontend tiene IP elástica y es la misma que utiliza la entrada `ADDRESS` acordada.
- ☐ El sitio funciona por el DNS asignado y HTTPS presenta un certificado válido.
- ☐ Apache frontend sirve los archivos estáticos y reenvía correctamente las solicitudes de la API.
- ☐ PHP procesa las solicitudes en el backend y la aplicación funciona con la base de datos de prueba.
- ☐ El grupo de seguridad frontend ya no permite entrada por TCP/3306; el backend solo acepta la API desde el grupo frontend.
- ☐ MySQL escucha solo en loopback; el grupo de seguridad no tiene una regla de entrada para el puerto 3306.
- ☐ El usuario de base de datos de la aplicación no es `root`, está limitado a la base necesaria y no permite conexiones remotas.
- ☐ La base de datos se ha migrado y verificado antes de retirar la instalación original del frontend.
- ☐ No hay contraseñas, claves privadas ni otros secretos en la entrega.

6. Entrega

Entregad una memoria breve con:

1. Diagrama de las dos instancias y del recorrido navegador → frontend → backend → MySQL.
2. Datos de la EC2 frontend reutilizada y AMI, tipo, región e IP privada del backend.

3. Tabla de reglas de ambos grupos de seguridad.
4. Resumen de la migración de la aplicación y la base de datos, y de la configuración de Apache, PHP y MySQL, sin revelar secretos.
5. Evidencias de que el DNS y HTTPS siguen funcionando, una operación de la aplicación y la configuración que demuestre que MySQL no es accesible desde la red.
6. Incidencias y estado de los recursos al finalizar. Conservad la EC2 frontend de la Práctica 1a mientras sea necesaria para mantener el DNS y la IP elástica.